Nama : Zahwa Agroli
NIM : 24/535089/TK/59335
Kelas : C - Teknik Biomedis

ASD - assigment

Problem 1

Snippet A

```
1 // n >= 1
2 int sum = 0 ;
3 For (int i=0 ; i<n ; i++) {
4     For (int j=0 ; j<i ; j++) {
5         sum += arr [i] + arr [j] ;
6     }
7 }
```

Hint : The inner loops runs 0, 1, 2, ..., n-1 times for each value of i. Use $\sum_{i=0}^{n-1} = \frac{n(n-1)}{2}$

line	Operation	execution	cost	total cost
2	sum = 0	1	1	1
3a	Outer init : i = 0	1	1	1
3b	outer cond : i < n	n + 1	1	n + 1
3c	outer incr : i++	n	2	2n
4a	inner init : j = 0	n	1	n
	(1 add + 1 assign)			
4b	inner cond : j < i	$\frac{n(n-1)}{2} + n$	1	$\frac{n(n-1)}{2} + n$
4c	inner incr : j++	$\frac{n(n-1)}{2}$	2	$\left(\frac{n(n-1)}{2}\right) 2$
5	sum += arr [i] + arr [j]	$\frac{n(n-1)}{2}$	5	$\frac{5n^2 - 5n}{2}$
	(2 add, 1 assign,			
	2 access)			

(i) Exact operation count T(n)

$$T(n) = 1 + 1 + (n+1) + 2n + n + \frac{n^2 - n}{2} + n + \left(\frac{n^2 - n}{2}\right)2 + \frac{5n^2 - 5n}{2}$$

$$= \frac{8n^2}{2} + 5n - \frac{8n}{2} + 3$$

$$= 4n^2 + n + 3$$

(ii) Big - O class

Asymptotic simplication : dropping the lower - order terms (0.5n + 3) and the constant factor (3.5) $\Rightarrow T(n) = 4n^2 + n + 3 = O(n^2)$

(iii) Best / Worst case

The best and worst cases are identical. The nested loops for i and j will always iterate until they reach limits defined by n, regardless of the values stored in the array arr. There are no break or return statements midway that could terminate the execution early or speed it up.

SIDU

Dipindai dengan CamScanner

Snippet B

```
1 //n >= 1 ; arr is a sorted array of n integers
2 int lo = 0, hi = n-1, target = 42 ;
3 bool found = false ;
4 while (lo <= hi) {
5     int mid = lo + (hi - lo) / 2 ;
6     if (arr [mid] == target)  { found = true ; break ; }
7     else if (arr [mid] < target) { lo = mid + 1 ; }
8     else                         { hi = mid - 1 ; }
9 }
```

line	Operation	execution	cost	total cost
2	int lo = 0, hi = n-1, target = 42	1	4	4
3	bool found = false	1	1	1
4	loop cond : lo <= hi	$([\log_2 n] + 1) + 1$	1	$([\log_2 n] + 1) + 1$
5	int mid = lo + (hi - lo) / 2	$([\log_2 n] + 1)$	4	$4([\log_2 n] + 1)$
6	if (arr [mid] == target)	$([\log_2 n] + 1)$	2	$2([\log_2 n] + 1)$
7	else if (arr [mid] < target)	$([\log_2 n] + 1)$	2	$2([\log_2 n] + 1)$
7/8	lo = mid + 1 or hi = mid - 1	$([\log_2 n] + 1)$	2	$2([\log_2 n] + 1)$

(i) Exact operation count ($T(n)$)

$$T(n) = 4 + 1 + ([\log_2 n + 1]) + 1 + 4([\log_2 n] + 1) + 2([\log_2 n] + 1) + 2([\log_2 n] + 1) + 2([\log_2 n] + 1) + 2([\log_2 n] + 1)$$

$$T(n) = 11[\log_2 n] + 17$$

(ii) Big-O Class

Asymptotic simplification : dropping the constant (11 and 17)

$$T(n) = 11[\log_2 n] + 17 = O(\log n)$$

(iii) Best / Worst case

The best and worst cases differ

- Best case : this occurs when the target is found exactly at the mid index during the very first iteration. The complexity is $O(1)$ because the loop terminates immediately after a single comparison.
- Worst case : this occurs when the target is not present in the array or is located at the final possible sub-division of the search space. The complexity is $O(\log n)$ because the search space is halved repeatedly until it is exhausted.

SiDU

Dipindai dengan CamScanner

Snippet C

```
1  // n >= 1
2  int count = 0;
3  for (int i = 0; i < n; i++) {
4      for (int j = i + 1; j < n; j++) {
5          for (int k = 0; k < n; k += 2) {    // note : k += 2
6              if (arr[i] != arr[j] && arr[k] > 0)
7                  count++;
8          }
9      }
10 }
```

Hint : The innermost loop runs $\lceil n/2 \rceil$ times regardless of i and j. The middle loop runs a total of $\frac{n(n-1)}{2}$ times (summed over all i)

line	Operation	Execution	Cost	Total Cost
2	int count = 0	1	1	1
3a	outer init : int = 0	1	1	1
3b	outer cond : $i < n$	$n+1$	1	$n+1$
3c	outer incr : i++	n	2	$2n$
4a	mid init : int $j = i + 1$	n	2	$2n$
4b	mid cond : $j < n$	$B + n$	1	$B + n$
4c	mid incr : j++	B	2	$2B$
5a	inner init . int $k = 0$	B	1	B
5b	inner cond : $k < n$	$B(K+1)$	1	$BK + B$
5c	inner incr : $k += 2$	BK	2	$2BK$
6	if (arr[i] != arr[j] && arr[k] > 0 (3 access, 2 comp, 1 logic)	BK	6	$6BK$
7	count++	BK	2	$2BK$

(i) Exact operation count ($T(n)$)

$$B = \frac{n(n-1)}{2} \quad ; \quad K = \frac{n}{2}$$

$$T(n) = 1 + 1 + n + 1 + 1n + 2n + (B + n) + 2B + B + (BK + B) + 2BK + 6BK + 2BK$$

$$= 3 + 6n + 5B + 11BK$$

$$= 3 + 6n + 5\left(\frac{n^2 - n}{2}\right) + 11\left(\frac{n^3 - n^2}{4}\right)$$

$$= 3 + 6n + \frac{5}{2}n^2 - \frac{5}{2}n + \frac{11}{4}n^3 - \frac{11}{4}n^2$$

$$T(n) = \frac{11}{4}n^3 - \frac{1}{4}n^2 + \frac{7}{2}n + 3$$

(ii) Big-O Class

Asymptotic simplification : dropping lower-order terms and the constant

$O(n^3)$

SiDU

Dipindai dengan CamScanner

(iii) Best / Worst case

- Best and worst case are identical asymptotically but differ operationally.
- Worst case ($O(n^3)$) : this occurs when the first condition in the if statement (arr [i] != arr [j]) is always true. Consequently, the program must evaluate the second condition (arr [k] > 0) and execute the count ++ operation.
- Best case ($O(n^3)$) : this occurs when the first condition arr [i] != arr [j] is always false. Due to the short-circuiting behavior of the logical && operator, the program skips the evalution of arr [k] > 0 and the count ++ increment.

SiDU

Dipindai dengan CamScanner

Problem 3 : Linear search vs Binary search

Part B - Complexity comparison table

For each cell, write the Big-O class and give a one-sentence explanation. Then answer : At what minimum array size n does binary search become more efficient than linear search in the worst case? Show your reasoning.

Operation	Best Case	Worst Case	Average Case
Linear search (unsorted)	O(1) Target ditemukan pada indeks pertama (i=0) sehingga loop langsung return.	O(n) Target tidak ada atau berada di indeks terakhir sehingga loop berjalan sebanyak n kali	O(n) Rata-rata target berada ditengah (indeks n/2), sehingga butuh n/2 iterasi → tetap O(n)
Linear search (sorted, early exit)	O(1) Target adalah elemen pertama atau elemen pertama sudah lebih besar dari target (berhenti pada iterasi pertama)	O(n) Target tidak ada dan nilainya lebih besar dari semua elemen, loop tetap harus berjalan penuh	O(n) Early exit menghemat waktu jika target tidak ada ditengah, tetapi secara asimptotik tetap O(n).
Binary search (sorted)	O(1) Target tepat berada dielemen tengah array sehingga langsung ditemukan pada probe pertama	O(log n) Target tidak ada atau diujung array (boundary) setiap iterasi membagi dua search space → maks $\log_2(n)+1$ iterasi	O(log n) Setiap probe mengeliminasi setengah elemen dan average depth of search tree = $\log_2 n$

Formula Worst-Case :

Algoritma	Worst-Case T(n)	Condition to be slower
Linear Search	$T_L(n) = n$ comparisons	slower when $n > \log_2(n)+1$
Binary Search	$T_B(n) = \text{floor}(\log_2(n))+1$ comparisons	slower when $\log_2(n)+1 > n$

Syarat binary search lebih efisien dari linear search pada worst case jika $\text{floor}(\log_2(n))+1 < n$

n	$T_L(n) = n$	$T_B(n) = \text{floor}(\log_2 n)+1$	Lebih efisien
1	1	1	sama
2	2	2	sama
3	3	2	binary lebih cepat
4	4	3	binary lebih cepat
10	10	4	binary lebih cepat
100	100	7	binary lebih cepat
1000	1000	10	binary lebih cepat

SiDU

Dipindai dengan CamScanner

Kesimpulan : nilai minimum n dimana search lebih cepat daripada linear search dalam worst case adalah n = 3. Karena pada linear search memerlukan hingga 3 comparions (worst case = check all 3), sementara itu, binary search hanya memerlukan floor ($\log_2 3$) + 1 = 2 comparions. Untuk n = 1 dan n = 2, kedua algoritma tersebut memerlukan jumlah perbandingan dalam worst case. Mulai dari n = 3 keatas, binary search secara asimtotik dan dalam praktiknya lebih efisien

SiDU

Dipindai dengan CamScanner

Problem 4

Consider the pushBack function from Problem 2, which doubles the capacity when ever size == capacity, starting from capacity = 2.

Part A (Trace)

Complete the table below by tracing 12 consecutive pushBack calls on an initially empty DynArray. For each call, record the value pushed, the resulting size, the resulting capacity, and the cost of that specific call (cost = 1 for normal append; cost = 1 + old size when a resize occurs).

Call #	Value pushed	size after	capacity after	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	3
4	40	4	4	1
5	50	5	8	5
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	9
10	100	10	16	1
11	110	11	16	1
12	120	12	16	1

Part B (total cost)

1. Sum all values in the Cost column. Call this T_{12}

$T_{12} = 1+1+3+1+5+1+1+1+9+1+1+1$

$T_{12} = 26$

2. What is the average cost per call, i.e. $T_{12}/12$?

$\frac{T_{12}}{12} = \frac{26}{12} \approx 2.17$

3. If we label the n-th call's cost c_n, write the full formula for the total copy work done across all resizes during n pushbacks. Identify this as a geometric series and evaluate it. What does this tell you about the amortised cost per call ?

Resizes occur when capacity doubles : at sizes 2, 4, 8, 16, 32, ... each resize at capacity c copies c elements. For n pushBack calls starting from capacity 2, resizes copy is

Copywork = 2 + 4 + 8 + 16 + ... up to the last resize $\leq n$.

A geometric series with first term $a = 2$, ratio $r = 2$, and at most $\log_2(n)$ terms :

Total copy work $= 2+4+8+\ldots+2^k$ where $2^k \leq n < 2^{k+1}$ $= \frac{2(2^k-1)}{(2-1)} = 2^{k+1} - 2 < 2n$

So, total copy work $< 2n$. Adding the n base append operations, the total cost of n pushBack calls $= n + 2n = 3n = O(n)$. Dividing by n, the amortized cost per call $= O(1)$ (constant time on average)

SiDU

Dipindai dengan CamScanner

Part C (Growth Factor Comparison)

Suppose instead of doubling the capacity, pushBack grows by adding a fixed amount $k = 4$ (i.e., new capacity = old capacity + 4)

1. Redo the trace from Part A (12 calls, starting from capacity 2) under this new strategy. Record the same four column.

Call#	Value pushed	size after	capacity after	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	6	3
4	40	4	6	1
5	50	5	6	1
6	60	6	6	1
7	70	7	10	7
8	80	8	10	1
9	90	9	10	1
10	100	10	10	1
11	110	11	14	11
12	120	12	14	1

2. Compare the total cost of both strategies

Strategy	Total Cost	Amortized / Call
Doubling (×2)	$1+1+3+1+5+1+1+1+9+1+1+1 = 26$	$\frac{26}{12} = 2.17$
Fixed increment (+4)	$1+1+3+1+1+1+1+7+1+1+1+11+1 = 30$	$\frac{30}{12} = 2.50$

The total cost of the fixed-increment strategy is higher than doubling strategy and more efficient for 12 operations.

3. For a general sequence of n pushbacks, derive $T(n)$ for both the doubling and the fixed-increment strategies. Show that doubling gives $O(n)$ total whereas fixed-increment gives $O(n^2)$ total. Express the amortised cost per call in each case.

-> Doubling Strategy

Resizes occur at sizes $2, 4, 8, 16, \ldots, 2^k \leq n$. Copy costs form a geometric series:

$T_{copy}(n) = 2 + 4 + 8 + \ldots + 2^k = 2^{k+1} - 2$ (sum of geometric series)

Since $2^{k+1} < 2n$, $T_{copy}(n) < 2n$

Total cost : $n + T(n) < n + 2n = 3n$

$= O(n)$

Amortized cost percall $= \frac{O(n)}{n} = O(1)$

-> Fixed increment Strategy

Starting capacity = 2, resizes occur at sizes $2, 6, 10, 14, \ldots (2 + 4i)$. Number of resizes $\approx n/4$.

Copy costs form an arithmatic series :

SiDU

Dipindai dengan CamScanner

$T_{copies}(n) = 2 + 6 + 10 + 14 + \dots + (\text{last resize} \approx n)$

Number of terms $m \approx n/4$

$$T_{copies}(n) = \frac{m}{2}(\text{first} + \text{last})$$

$$\approx \frac{n/4}{2}(2 + n)$$

$$= \frac{n}{8}(n + 2)$$

$$= \frac{n^2}{8} + \frac{n}{4} \Rightarrow O(n^2)$$

Total cost

$$T_{total}(n) = n + O(n^2)$$

$$= O(n^2)$$

Amortized cost per call : $\frac{O(n)^2}{n} = O(n)$

SiDU

Dipindai dengan CamScanner